โครงการอบรมโอลิมปิกวิชาการ ค่าย 2 สาขาคอมพิวเตอร์
วันที่ 27 มี.ค. – 10 เม.ย. 2569

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเชียงใหม่

การวิเคราะห์ความซับซ้อนของขั้นตอนวิธี Big-Oh

วันที่ 27 มีนาคม 2569 (เช้า)

ผู้ช่วยศาสตราจารย์เบญจมาศ ปัญญางาม

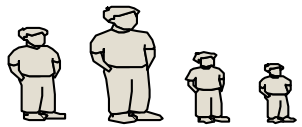
อัลกอริทึม

An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.

- **Computational problem**
  - ✓ **Sorting, Searching, Partitioning**
  - ✓ **Matrix multiplication, Prime number testing, Random number generation**
  - ✓ **Minimum spanning tree, Graph coloring**
- **ตัวอย่าง : ปัญหาการเรียงลำดับข้อมูล**
  - ✓ **Input: ชุดลำดับข้อมูล n ตัว (a1, a2, …, an)**
  - ✓ **Output: ชุดข้อมูลที่มีการเรียงสับเปลี่ยนเพื่อจัดลำดับใหม่ (a1, a2, .. , an)**

โดยที่ a1 ≤ a2 ≤ … ≤ an)

ตัวอย่าง (Instance) สำหรับปัญหานี้



Input

5 9 1 3

Sorting Algorithm

Output

1 3 5 9

ความสำคัญของการวิเคราะห์อัลกอริทึม

- ✓ ปัญหาหนึ่ง ๆ จะมีหลายขั้นตอนวิธีในการแก้ปัญหา ควรเลือกจัดเก็บแบบ/วิธีไหนดี ?
  - เช่น จัดเก็บข้อมูลในลักษณะ **List, Stack, Queue** ใช้เวลาในการลบข้อมูลต่างกัน
    เลือกวิธี **Insertion Sort** หรือ **Merge Sort** ใช้เวลาในการเรียงลำดับข้อมูลต่างกัน

- ✓ สิ่งที่คำนึงถึงในการออกแบบ อัลกอริทึม ?

"Correctness" : **for all possible, it halts with the correct output**

"Efficient Algorithm" : **Analyzing algorithm to predict *resource utilization***

1) **Memory** : **Space complexity**
2) **Running time** : **Time complexity (*focus)**

การเลือกอัลกอริทึมที่มีประสิทธิภาพ (Efficient Algorithm)

1) **Space complexity :** ปริมาณพื้นที่หน่วยความจำ(Memory) ที่ใช้

เช่น กรณีการเก็บเลขจำนวนเต็ม 1 ค่า ใช้พื้นที่ 4 bytes

int a,b,c ➔ 4 bytes * 3 = 12 bytes

int arr[100] ➔ 4 bytes *100 = 400 bytes

2) **Time complexity :** เวลาที่ใช้ในการทำงานของโปรแกรม

➢ การเปรียบเทียบ อาจต้องเลือกระหว่างการใช้หน่วยความจำและประสิทธิภาพของ Runtime

✓ เช่น ระหว่าง Merge sort และ Bubble sort เมื่อ n คือขนาดข้อมูลนำเข้า (input size)

	Merge sort	Bubble sort
1) ใช้พื้นที่	2n x sizeof(type)	n x sizeof(type)
2) เวลา	n long n	n^2

"The more time efficiency you have, the less space efficiency you have and vice versa"

การเลือกอัลกอริทึมที่มีประสิทธิภาพ (Efficient Algorithm)

การวิเคราะห์อัลกอริทึม : Time complexity โดยเปรียบเทียบ Running time ที่อัลกอริทึมใช้ในการทำงาน

- Most algorithms transform input objects into output objects.
- ดังนั้น Running time คือ เวลาที่ใช้ในการทำงานของอัลกอริทึมเพื่อเปลี่ยน input เป็น output ที่ต้องการ

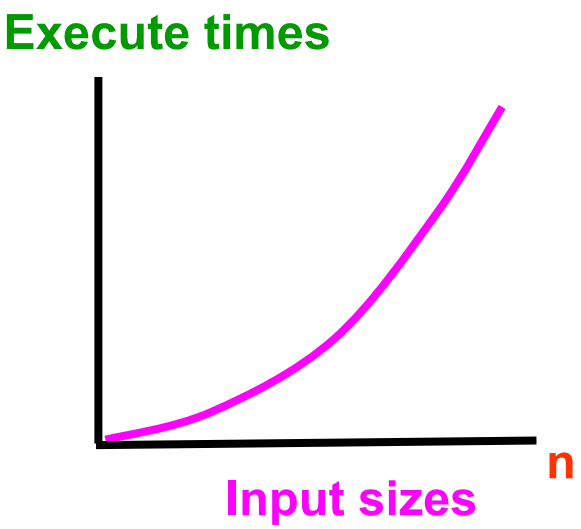
<u>วิธีการ</u>

1) คำนวณ Execution times : ทำได้ยากและเวลาขึ้นอยู่กับประสิทธิภาพของ Hardware

2) นับ Number of statements executed: จำนวนคำสั่งขึ้นกับรูปแบบไวยากรณ์ (Syntaxes) ของภาษาที่ใช้ในการเขียนโปรแกรม

การวิเคราะห์อัลกอริทึม : Time complexity

- **Running time คือ** เวลาที่ใช้ในการทำงานของอัลกอริทึมเพื่อเปลี่ยน input เป็น output ที่ต้องการ
  - เวลาขึ้นกับขนาดของ input (ค่า n)



1) คำนวณ Execution times : ปัญหาที่เกิดขึ้น

- เขียนโปรแกรม แล้วทดสอบกับ input ขนาดต่าง ๆ (n) เพื่อบันทึกเวลาที่ใช้ในการทำงาน จึงทำได้ยาก
- อาจไม่ใช่เวลาจริงสำหรับบางกลุ่ม input ที่ไม่ได้นำมาทดสอบ
- การเปรียบเทียบระหว่าง 2 อัลกอริทึมได้ยาก เพราะขึ้นกับ Hardware ที่ใช้

การวิเคราะห์อัลกอริทึม : Time complexity

➢ **Running time คือ** เวลาที่อัลกอริทึมใช้ เพื่อเปลี่ยน input เป็น output ที่ต้องการ

✓ เวลาขึ้นกับขนาดของ input (ค่า n)

```
sum = 0;
for (i =0; i< n; i++)
    sum = sum + A[i];
```

2) Number of statements executed :

- คำนวณเวลาด้วยการนับจำนวนคำสั่งที่ทำงาน
- จำนวนครั้งที่คำสั่งทำงานขึ้นอยู่กับจำนวนข้อมูล

ถ้าอาร์เรย์ A เก็บข้อมูล 5 จำนวน (n =5)

	จำนวนครั้ง
sum = 0	1
i = 0	1
i< n	6 ➔ n + 1
i++	5*2 (บวก และกำหนดค่า) ➔ 2n
sum=sum+A[i]	5*3 (หาตำแหน่ง array, บวก และกำหนดค่า➔ 3n

รวมทั้งหมด T(n) = 1+ 1 + n+1 + 2n + 3n

= 6n + 3

"Characterizes running time of an algorithm as a function of the input size, *n*"

Time complexity = T(n)

การวิเคราะห์อัลกอริทึม : สรุป Time complexity

- **Running time คือ** เวลาที่อัลกอริทึมใช้ เพื่อเปลี่ยน input เป็น output ที่ต้องการ
  - ✓ เวลาขึ้นกับขนาดของ input (ค่า n)

Problem :

จะหาวิธีวิเคราะห์หา Running times อย่างไร ไม่ให้ขึ้นกับฮาร์ดแวร์และซอฟต์แวร์ที่ใช้ ?

"an analysis is independent of machine time, programming style, etc."

วิธี 1) Execution times : เวลาที่ใช้ขึ้นกับ Hardware

วิธี 2) Number of statements executed : จำนวนคำสั่งขึ้นกับรูปแบบไวยากรณ์ (Syntaxes) ของภาษาและซอฟต์แวร์ที่ใช้เขียนโปรแกรม

การวิเคราะห์อัลกอริทึม : Time complexity T(n) : Big O

Solution : หา **Growth rates of the functions** แล้วเขียนในรูป **Big O notation**

"Big O notation gives us an algorithm's complexity in terms of input size, N"

ขั้นตอนการหา Time complexity ของอัลกอริทึม

1. นับจำนวน primitive operations (นับจาก Pseudocode แทน โดยไม่ขึ้นกับภาษาที่ใช้)
2. เขียนในรูปฟังก์ชันของขนาด input ➔ T(n)
3. หา **Growth rates of the functions** : เราสนใจในกรณี **Very large input size (n)**

✓ **Drop lower order terms**

✓ **Drop all the constant multipliers**

Time complexity Comparison : Big-Oh Notation

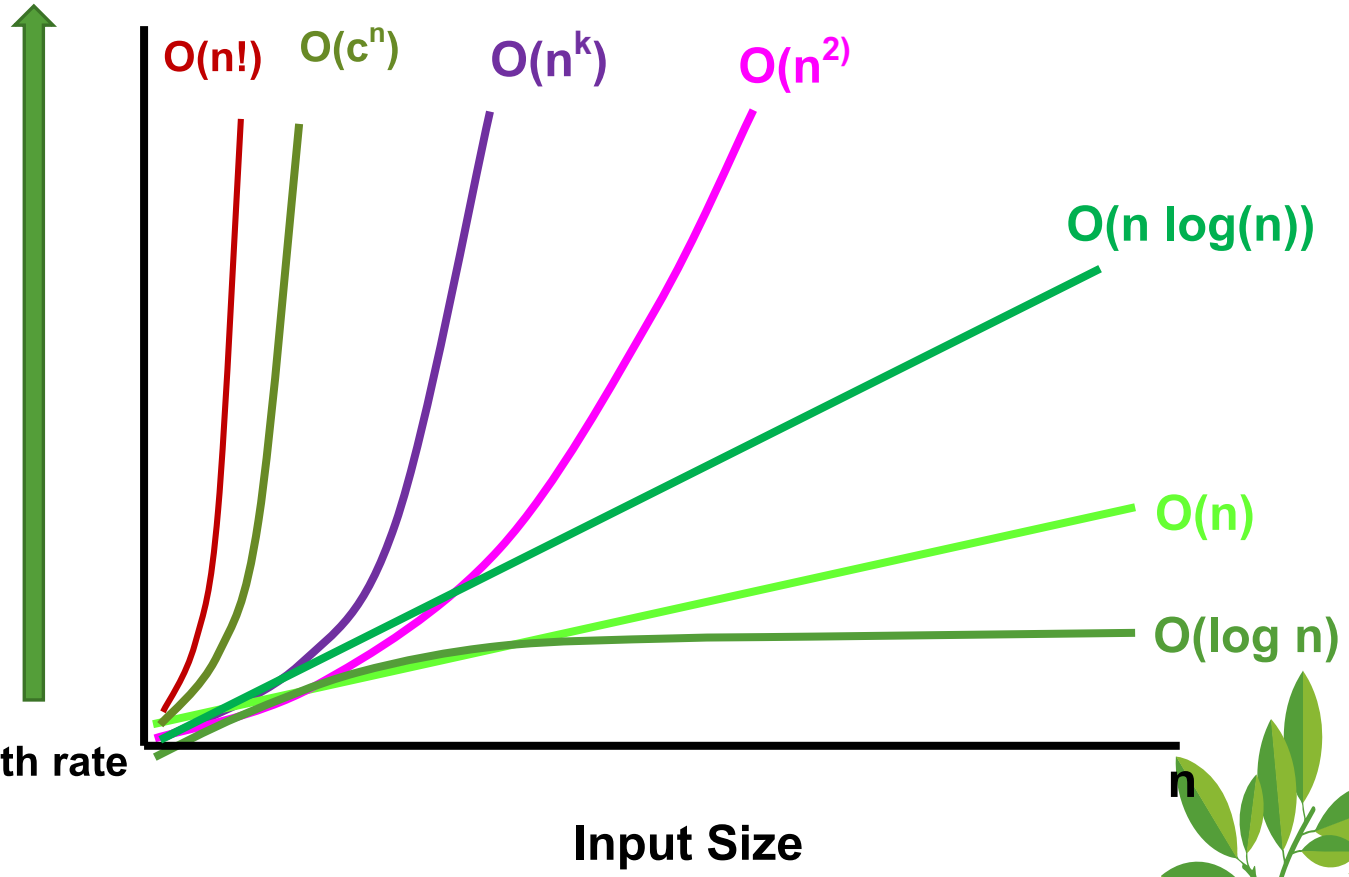
Growth the functions

- Constant = a
- Log ≈ log(n)
- Linear ≈ n
- Log-linear ≈ n $\log_b$ n
- Quadratic ≈ n^2
- Cubic ≈ n^3
- Polynomial ≈ n^k
- Exponential ≈ a^n , 2^n, 3^n , ….
- n Factorial = n!

Efficient time

Running time complexity

(Big-O)



Big-Oh Notation

- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$

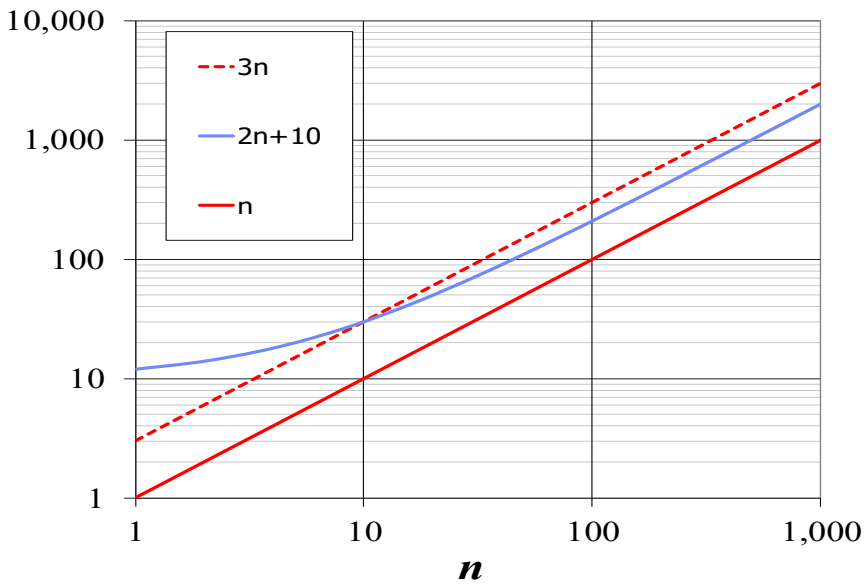
if there are positive constants c and n_0 such that

$$f(n) \le cg(n) \quad \text{for } n \ge n_0$$

- <u>ตัวอย่าง 1</u> :f(n) = $2n + 10$ is $O(n)$

จะมีค่า $c > 0$ และ $n_0 \ge 1$ ที่ทำให้

สมการ $2n + 10 \le cn$ เป็นจริงสำหรับทุกค่า $n \ge n_0$



ลำดับการแก้สมาการ

$2n + 10 \le cn$

$(c - 2)\, n \ge 10$

$n \ge 10/(c - 2)$

เลือก $c = 3$ และ $n_0 = 10$

ตัวอย่าง Big-Oh Notation

- <u>ตัวอย่าง 2</u> : f(n) = $7n$-2 is $O(n)$

  จะมีค่า $c > 0$ และ $n_0 \geq 1$ ที่ทำให้

  สมการ $7n-2 \leq cn$ *เป็นจริง* สำหรับทุกค่า $n \geq n_0$

  ตัวอย่างคำตอบ คือ c = 7 และ $n_0 = 1$

- <u>ตัวอย่าง 3</u> : f(n) = $3n^3 + 20n^2 + 5$ is $O(n^3)$

  จะมีค่า $c > 0$ และ $n_0 \geq 1$ ที่ทำให้

  สมการ $3n^3 + 20n^2 + 5 \leq cn^3$ เป็นจริงสำหรับทุกค่า $n \geq n_0$

  ตัวอย่างคำตอบ คือ c = 4 และ $n_0 = 21$

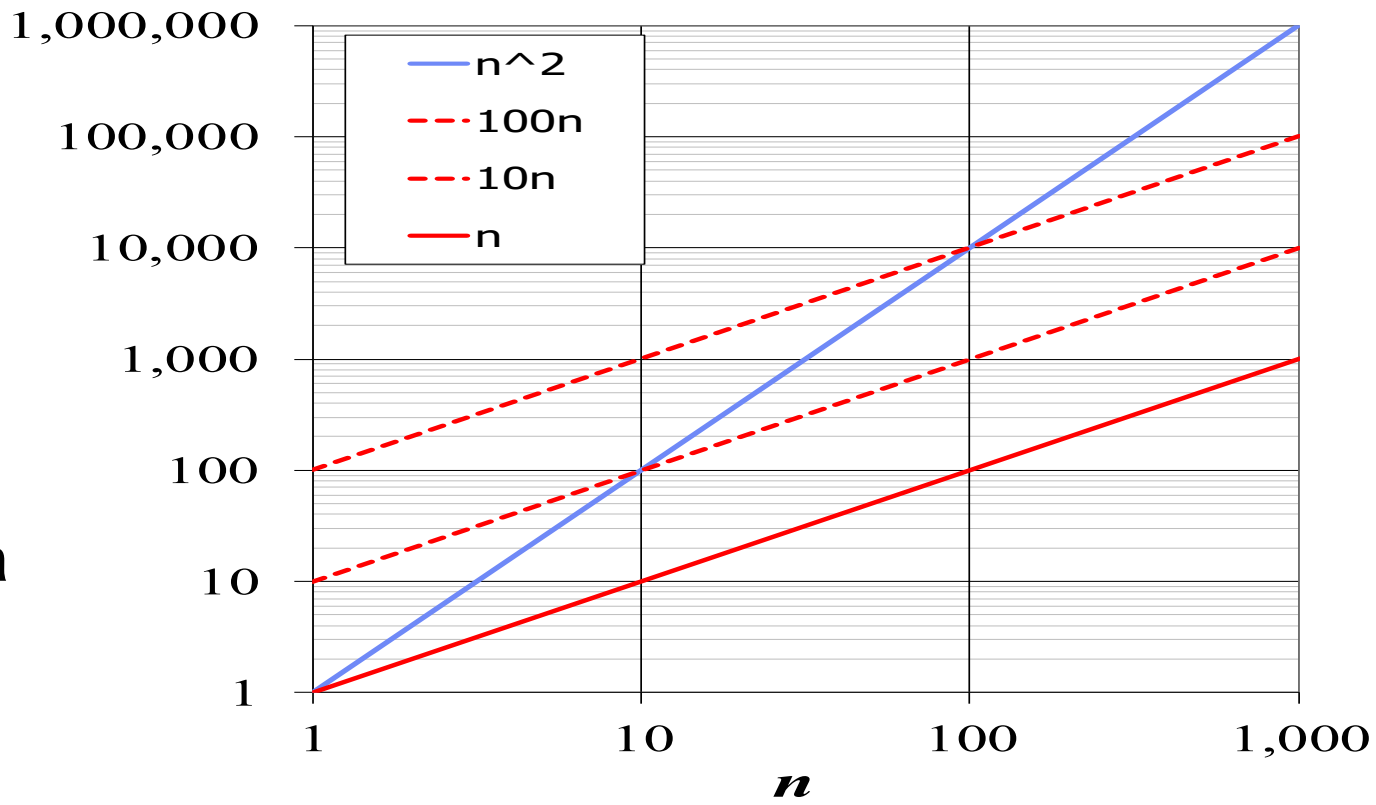
Big-Oh Notation

ฟังก์ชัน f(n) = n^2 is not $O(n)$

จาก $n^2 \leq cn$

$n \leq c$

The above inequality cannot be satisfied since *c* must be a constant



ขั้นตอนการวิเคราะห์ T(n) = O(?)

ขั้นตอนการหา Time complexity ของอัลกอริทึม

```
int SumAll(int A[], int n) {
    //หาผลรวมข้อมูลในอาร์เรย์ A ขนาด n
    int i, sum = 0;
    for (i =0; i< n; i++)
        sum = sum + A[i];
    return(sum);
}
```

1. นับจำนวน primitive operations

กรณี n = 5	กรณี n ใด ๆ
1	1
1+ 6 + 5*2	1+ n +1+ 2n= 3n+2
5+5+5	3n
1	1

2. เขียนฟังก์ชัน T(n) = 6n + 4

3. หา Growth rates of the functions

T(n) = ~~6~~n ~~+ 4~~ = O(n) ➔ Running time complexity in Big-O term

- Drop lower order terms
- Drop all the constant multipliers

ขั้นตอนการวิเคราะห์ T(n) = O(?)

ตัวอย่างการหา Growth rates of the functions

- Drop lower order terms
- Drop all the constant multipliers

ดังนั้น ถ้า T(n) เป็น polynomial of degree d,

จะสรุปว่า T(n) is $O(n^d)$,

เช่น $2n^3 + 10^5 n^2 + 10^8 n$

$\cancel{2}n^3 \cancel{+ 10^5 n^2 + 10^8 n} = O(n^3)$

$T(n) = 3n^2 + 7n - 2$

$T(n) = \cancel{3}n^2 + \cancel{7n - 2}$

$= O(n^2)$ → Big O notation

การวิเคราะห์ T(n) = O(?) ในกรณีต่าง ๆ

1) Loop : จำนวนครั้งขึ้นกับ input size / number of repetitions

```
for (i=1; i<n; i++) {
  x = y + z ;
}
```

n times

constant time = c

T(n) = n x c

T(n) = ~~c~~n

= O(n)

T(n) = n x 1

T(n) = n

= O(n)

หา Growth rates of the functions

- Drop lower order terms
- Drop all the constant multipliers

การวิเคราะห์ T(n) = O(?) ในกรณีต่าง ๆ

2) Sequential Statements

```
x = 10;
```
constant time = c_1

```
x = x + a;
```
constant time = c_2

```
for (i=1; i<n; i++) {
   x = y + z;
}
```
$c_3 n \rightarrow n$

```
for (i=1; i<n; i++) {
   x = y + z ;
}
```
$c_4 n \rightarrow n$

```
for (i=1; i<n^2; i++) {
   x = y + z ;
}
```
$c_5 n^2 \rightarrow n^2$

$$T(n) = c_1 + c_3 n + c_4 n + c_5 n^2$$

$$T(n) = \cancel{c_1 + c_2 + (c_3 + c_4)n} + c_5 n^2$$

$$= O(n^2)$$

หรือ $T(n) = n^2 \cancel{+ 2n + 2} = O(n^2)$

หา Growth rates of the functions

- Drop lower order terms
- Drop all the constant multipliers

การวิเคราะห์ T(n) = O(?) ในกรณีต่าง ๆ

3) Nested Loop

```
for (i=1; i<n; i++) {
    for (k=1; k < n ; k++)
        x = y + z;
}
```

n times

n times

constant time

$T(n) = (n \times (n \times c)) = cn^2$

$T(n) = \cancel{c}n^2$

$= O(n^2)$

หรือ $T(n) = (n \times (n \times 1)) = n^2$

$= O(n^2)$

หา Growth rates of the functions

- Drop lower order terms
- Drop all the constant multipliers

การวิเคราะห์ T(n) = O(?) ในกรณีต่าง ๆ

4) If-else statements :

```
if (condition){
  do somethings ……
}
```

$cn \rightarrow n$

$\rightarrow T(n) = O(n)$

```
if (condition){
  do somethings ……
}
else{
  do  somethings…
}
```

$c_2n+c_1 \rightarrow n$

$c_3n^2+c_2n+c_1 \rightarrow n^2$

$\rightarrow T(n) = \max(n, n^2)$

$= O(n^2)$

ถ้ามีหลายทางเลือก

- ให้เลือกทางเลือกที่ใช้เวลาสูงที่สุด

ตัวอย่างการวิเคราะห์ T(n) = O(?)

```
int Function2(const int X[], int n, int A[] ){
    int i, j, s
/* 1*/ for (i = 0; i < n; i++)
/* 2*/     s = X[0];
/* 3*/     for (j = 1; j <= i; j++) {
/* 4*/         s += X[ j ];
/* 5*/     A[i] = s/(i+1);
/* 6*/ return 0;
}
```

$T(n)=n^2+1 = O(n^2)$

$\rightarrow 1$

$\sum_{j=1}^{i} 1 = 1+1+..+1 \rightarrow i$

$\rightarrow 1$

$\rightarrow 1$

$= i + 2$

$\sum_{i=1}^{n} i = 1 + 2+\ldots+(n-1)$

$= (n-1)n/2$

$= n^2$

$\rightarrow 1$

ตัวอย่างการวิเคราะห์ T(n) = O(?)

$T(n) = n^2 + 2 = O(n^2)$

```
int Function3(const int A[ ], int N ) {
/* 1*/int i , j, sum, MaxSum = 0;        → 1
/* 2*/for (i = 1; i <= n; i++){
/* 3*/    sum = 0;                       → 1
/* 4*/    for (j = i ; j <= n; j++)
/* 5*/        sum += A[ j ];             → 1
/* 6*/    if (sum>MaxSum )               → 1
/* 7*/        MaxSum = sum;
/* 8*/ }
/* 9*/ return MaxSum;                    → 1
}
```

$$\sum_{j=i}^{n} 1 = 1+1+\ldots\ldots+1$$

$= n-i+1$

$= n+i+1$

เลือกกลุ่มคำสั่งที่ใช้เวลามากสุด

$$\sum_{i=1}^{n} n-i+1$$

$= n+(n-1)+..+1$

$= n^2$

ตัวอย่างการวิเคราะห์ T(n) = O(?)

T(n)= n log n+ 2 = *O*(n log n)

```
int Function4(int n)
{ int  sum,i,j,k;
  /*1*/  sum=0;          ➔ 1
  /*2*/  for (i = n; i >0 ; i = i/2)
  /*3*/      for (j = 1; j<= n; j++)
  /*4*/          sum++;  ➔ 1
  /*5*/  return sum;     ➔ 1
}
```

$\sum_{j=i}^{n} 1$ =1+1+…+1 = n

= n+n+…..+n

= K * n

=(log n) n

= n log n

การหาค่า K

รอบที่ 1 i = n

รอบที่ 2 i = n/2

รอบที่ 3 i = n/4

รอบที่ 4 i = n/8

…

รอบที่ K i = $n/2^{k-1}$

(รอบสุดท้าย i จะเท่ากับ 1)

ดังนั้น 1 = $n/2^{K-1}$

2^{K-1}= n

K - 1 = log n

K = log n ~~+ 1~~

ดังนั้น K ➔ log n

แบบฝึกหัดการวิเคราะห์ T(n) = O(?)

T(n)= ……+ 1 = *O*(……)

```
int Function5(int A[], int n)
/* 1*/ for (i = n-1; i > 0; i--){
/* 2*/   j=0
/* 3*/   for (k = 1; k < i; k++)
/* 4*/     if (A[j] < A[k]  )  j = k;
/* 5*/   swap(A, i , j); //เรียกฟังก์ชันเพื่อสลับค่าในอาร์เรย์
/* 6*/ return 0;
}
```

→ 1

→ 1

$$\sum_{k=1}^{i} 1 = 1+1+..+1 = i$$

→ 1

= i

*เลือกกลุ่มคำสั่งที่ใช้เวลามากสุด

$$\sum_{i=n-1}^{1} i$$

= …… +…+ ……

=

→ 1

การวิเคราะห์ Time complexity T(n)

- Running times มี 3 กรณี : Best, Average, Worst
- เราสนใจเฉพาะ Worst case running time
  - วิเคราะห์ฟังก์ชัน T(n) ได้ง่าย

```
int FindPos((int A[], int n, int x) {
//ค้นหาว่าค่า x อยู่ในตำแหน่งใดในอาร์เรย์ A ขนาด n
    for (i =0; i< n; i++)          → c1n
        if (x == A[i])             → c1
            return (i);
    return(-1);                    → c2
}
```

Best case : O(1)

กรณีค้นเจอที่ตำแหน่งแรก

Average case: O(n)

กรณีค้นเจอในตำแหน่งอื่น ๆ ด้วยโอกาสพอ ๆ กัน

Worst case: O(n)

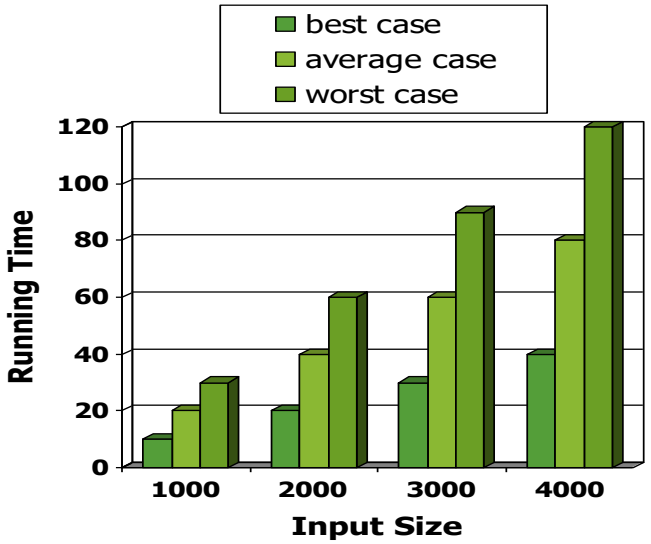
กรณีค้นหาไม่เจอ หรือที่ตำแหน่งท้าย

$T(n) = c_1 n + c_2 = O(n)$

การวิเคราะห์ Time complexity T(n)

Time complexity comparison of sorting algorithms

	Best	Worst	Average
• Selection Sort	n^2	n^2	n^2
• Bubble Sort	n	n^2	n^2
• Insertion Sort	n	n^2	n^2
• Merge Sort	n log n	n log n	n log n
• Quick Sort	n log n	n^2	n log n
• Heap Sort	n log n	n log n	n log n



Recursive Function

Recursive Function

ประกอบด้วย 2 ส่วนหลัก

Base Case: กรณีที่หยุดการเรียกใช้ฟังก์ชัน หรือ ไม่ทำการเรียกซ้ำอีก

Recursive Case: กรณีที่ฟังก์ชันเรียกใช้ตัวเองซ้ำ ๆ จนกว่าจะถึง base

```
if(test for base case)
        return some base case value
else if(test for another base case)
        return some other base case value
else //recursive case
        return (some work and then recursive call)
```

```
void Func1(int i) {
    if(i<3) {
        Func1(i+1);
        printf("%d", i);
    }
}
```

```
void Func1(int i) {
    if(i>=3) {
    }
    else{
        Func1(i+1);
        printf("%d", i);
    }
}
```

Recursive Function

- Recursive Function จะทำงานเพื่อแก้ปัญหาด้วยการเรียกตัวเองหลายครั้ง
- แต่ละครั้งที่จะเรียกตัวเอง จะสำเนาตัวแปร local และพารามิเตอร์ต่าง ๆ ของฟังก์ขณะเก็บไว้ใน stack memory เพื่อให้สามารถกลับมาทำงานต่อได้
- สิ่งสำคัญ
  - ✓ ควรตรวจสอบให้แน่ใจ ว่า จะมีกรณีสิ้นสุดการเรียกใช้ฟังก์ชัน (เข้า base case) เพื่อหลีกเลี่ยงการเรียกซ้ำไม่สิ้นสุด

```
void main(void){
    Func1(0);
}
```

```
void Func1(int i)
{
  if(i<3) {
    Func1(i+1);
    printf("%d", i);
  }
}
```

0

```
void Func1(int i)
{
  if(i<3) {
    Func1(i+1);
    printf("%d", i);
  }
}
```

1

```
void Func1(int i)
{
  if(i<3) {
    Func1(i+1);
    printf("%d", i);
  }
}
```

2

```
void Func1(int i)
{
  if(i<3) {
    Func1(i+1);
    printf("%d", i);
  }
}
```

```
#include<stdio.h>
void Func1(int i) {
    if(i<3) {
        Func1(i+1);
        printf("%d", i);
    }
}
void main(void) {
    Func1(0);
}
```

Output : 2 1 0

Recursive Algorithm Example

Example: *n factorial*

Ex. 5! = 5 * 4 * 3 * 2 * 1

4! = 4 * 3 * 2 * 1

3! = 3 * 2 * 1

2! = 2 * 1

และ 1! = 1 , 0! = 1

ดังนั้น ➔

5! = 5 * 4!

4! = 4 * 3!

3! = 3 * 2!

2! = 2 * 1!

1! = 1 , 0! = 1 base case

ใช้กรณีนี้เป็น baes case สำหรับเงื่อนไขที่หยุดการเรียกซ้ำ

กำหนด Recurrence Relation สำหรับการคำนวณ n factorial (n!) โดยเขียนฟังก์ชัน FAC(n) เป็นสมการต่อไปนี้

$$FAC(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ n * FAC(n-1) & \text{if } n > 1 \end{cases}$$

base case

Recursive Algorithm Example

Examples : n factorial (n!)

```
long FAC(int n) {
    if (n <= 1)
        return 1;
    else
        return n*FAC(n-1);
}
```

Condition that stop the recursion

Calling itself

จากสมการ Recurrence Relation :

$$FAC(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ n * FAC(n-1) & \text{if } y > 0 \end{cases}$$

```
void main() {
    int num;
    printf("Enter number  (n) : ");
    scanf("%d",&num);
    printf("n! = %ld", FAC(num));
}
```

Example of function operations

FAC(4)

4 * FAC(3)

4 * 3 * FAC(2)

4 * 3 * 2*FAC(1)

4 * 3 * 2*1

Example of Output

Enter number (n) : 4

n!=24

Recursive Algorithm Example

➢ **Examples : Power(x,y) ➔ x^y**

Ex. 2^4 , x =2,y =4

$2^4 = 2 * 2 * 2 * 2$

$2^3 = 2 * 2 * 2$

$2^2 = 2 * 2$

$2^1 = 2$

$2^0 = 1$

So that ➔

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1$

$2^1 = 2 * 2^0$

$2^0 = 1$ ← base case

กำหนด Recurrence Relation สำหรับการคำนวณ x^y โดยเขียนฟังก์ชัน Power(x,y) ด้วยสมการ

$$\text{Power}(x,y) = \begin{cases} 1 & \text{if } y = 0 \\ x * \text{Power}(x,y-1) & \text{if } y > 0 \end{cases}$$

← base case

Where x, y and results of power(x,y) are the integer numbers

Recursive Algorithm Example

จากสมการ Recurrence Relation :

$$\text{Power}(x,y) = \begin{cases} 1 & \text{if } y = 0 \\ x*\text{Power}(x,y-1) & \text{if } y > 0 \end{cases}$$

```
long power(int x, int y) {
    if (y == 0)  return(1);
    else  return(x*power(x, y-1));
}
```

condition that stop the recursion

Calling itself

```
void main() {
   int x, y;
   printf("Input x: ");scanf("%d", &x);
   printf("Input y: ");scanf("%d", &y);
   printf("Output : %ld", power(x, y));
}
```

Example of function operations

Input x: 3

Input y: 4

Output: 81

Recursion vs. Iteration

Recursion	Iteration
Recursion ends when base case become true	Loop ends when control variables 's value satisfies the condition
Each function call consumes any extra memory space (stack)	No extra space In each iteration
Infinite recursion may cause stack overflow error (memory full)	infinite loops (repeat forever without stopping) uses CPU cycles (not create extra memory)
Many of the problems can be solved easily using recursion if you think recursively.	

- recursive algorithm ใด ๆ ก็ตาม สามารถเขียนในลักษณะ Iteration ได้
  - ✓ แต่อาจจำเป็นต้องใช้โครงสร้างข้อมูล Stack เข้ามาช่วยในการทำงาน

```
int powerxy(int x,y) {
   long powxy = 1;

   while (y >= 1) {
      powxy *= X;
      --y;
   }
   return powxy;
}
```

```
long factorial(int n) {
   long fact = 1;

   while (n >= 1) {
      fact *= n;
      --n;
   }
   return fact;
}
```

การวิเคราะห์ Time complexity สำหรับ Recursive Algorithm

- **Running time จะคำนวณจากสมการ recurrence ของค่า T(n)**

1. t_A = เวลาในการทำงานส่วนของ base case
2. t_B = เวลาในการทำงานส่วนการเรียกตัวเอง (recursive) หรือเวลาของการแก้ปัญหาย่อย
3. t_C = เวลาในการทำงานอื่น ๆ ที่ไม่ใช่เวลาของการแก้ปัญหาย่อย หรือไม่ใช่ recursive

```
Selection_Sort_Recursive(A)
1.if (n <=1) return 1
2.j= FindIndexMax(A[1..n])
3.swap(A,n,j);
4.Selection_Sort_Recursive(A[1..n-1])
```

$$T(n)=\begin{cases} t_A & \text{กรณี base case} \\ t_B + t_C & \text{กรณี อื่น ๆ} \end{cases}$$

การวิเคราะห์ Time complexity สำหรับ Recursive Algorithm

- **Running time จะคำนวณจากสมการ recurrence ของค่า T(n)**

Selection_Sort_Recursive(A)

1.	if (n <=1) return 1	1
2.	j= FindIndexMax(A[1..n])	n
3.	swap(A,n,j);	1
4.	Selection_Sort_Recursive(A[1..n-1])	T(n-1)

1) t_A =T(1)= 1

2) Recursive call :

t_B =T(n-1)

3) ไม่ใช่ recursive call:

t_C= n+1 ➔ n

Recurrence Relation of time complexity

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ \underbrace{T(n-1)}_{t_B} + \underbrace{n}_{t_C} & \text{if } n > 1 \end{cases}$$

การวิเคราะห์ Time complexity สำหรับ Recursive Algorithm

- **Running time จะคำนวณจากสมการ recurrence ของค่า T(n)**

$$T(n)=\begin{cases} O(1) & \text{if } n=1 \\ T(n-1)+O(n) & \text{if } n>1 \end{cases}$$

หรือ

$$T(n)=\begin{cases} c & \text{if } n=1 \\ T(n-1)+cn & \text{if } n>1 \end{cases}$$

หรือ

$$T(n)=\begin{cases} c & \text{if } n=1 \\ T(n-1)+n & \text{if } n>1 \end{cases}$$

หรือ**

$$T(n)=\begin{cases} 1 & \text{if } n=1 \\ T(n-1)+n & \text{if } n>1 \end{cases}$$

มีความหมายเดียวกันทั้งหมด

การวิเคราะห์ Time complexity สำหรับ Recursive Algorithm

- **Running time จะคำนวณจากสมการ recurrence ของค่า T(n)**

```
BinarySearchRecursive(A[left..right])
1.  if (left > right) return(-1)
2.  m = (left + right)/2
3.  if x == A[m] return m;
4.  if x < A[m]
5.     return(BinarySearchRecursive (A[left..m-1]))
6.  else
7.     return(BinarySearchRecursive (A[m...right]))
```

1) $t_A = T(1) = 1$

2) Recursive call : $t_B = T(n/2)$

3) ไม่ใช่ recursive call: $t_C = 1$

Recurrence Relation of time complexity

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ \underbrace{T(n/2)}_{t_B} + \underbrace{1}_{t_C} & \text{if } n > 1 \end{cases}$$

การวิเคราะห์ Time complexity สำหรับ Recursive Algorithm

- **Running time จะคำนวณจากสมการ recurrence ของค่า T(n)**

```
Merge_Sort(A,p,r)
1. if p < r then
2.     q= ⌊(p+r)/2⌋
3.     Merge_Sort(A,p,q)
4.     Merge_Sort(A,q+1,r)
5.     Merge(A,p,q,r)
```

```
Merge(A,p,q,r)
1. i = p, j = q + 1, n = r - p + 1
2. for k = 1 to n
3.    if ((A[i]< A[j]) or (j > r)) and (i≤q)
4.        B[k] = A[i]
5.        i = i +1
6.    else
7.        B[k] = A[j]
8.        j = j +1
9. for k = 0 to n – 1
10.    A[p + k] = B[k]
```

1) $t_A = T(1) = O(1)$

2) Recursive call :

$t_B = 2T(n/2)$

3) ไม่ใช่ recursive call:

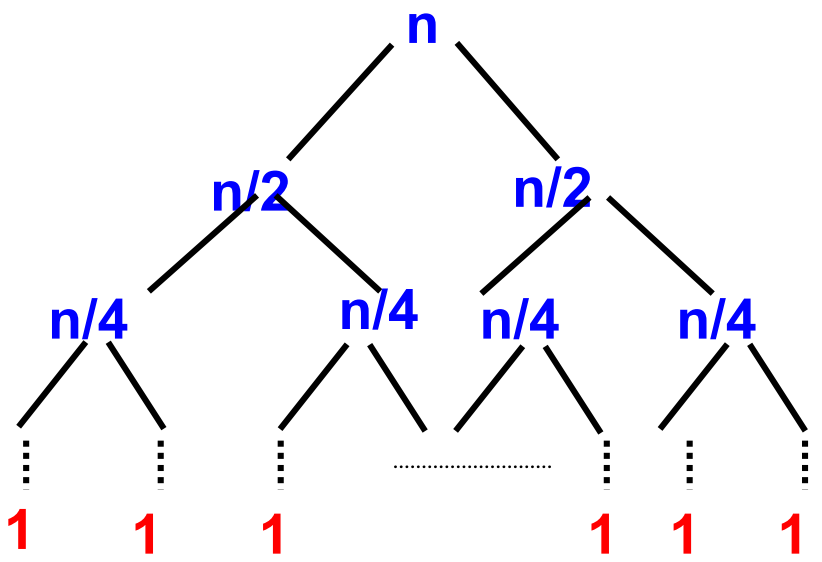
$t_C = n$

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

Recurrence Relation of time complexity

สมการ Recurrence 2T(n/2) + n, T(1)= 1 ตัวอย่าง

level	*จำนวนโหนด*	*เวลาแต่ละ level*
0 , T(n)	→ 1	1 x n → n
1 , T(n/2)	→ 2	2xn/2 → n
2 , T(n/4)	→ 4	4xn/4 → n
....		....
i=k, T(1)=T($n/2^k$)	→ 2^k	2^k xT(1) = $2^{\log_2 n}$ x 1 = $n^{\log_2 2}$ → n2



- วิธีหาจำนวน level (หาค่า k)

  รอบสุดท้ายคือ T(1) = T($n/2^k$) ดังนั้น n = 2^k → k = $\log_2 n$

- วิธีหาจำนวนโหนด
  - แต่ละ level ใน tree, level i, จะมีจำนวน 2^i โหนด
  - รอบสุดท้ายจะมี 2^k โหนด → $2^{\log_2 n}$

T(n) = ผลรวมเวลาในทุก level

= n + n + n + …… +n

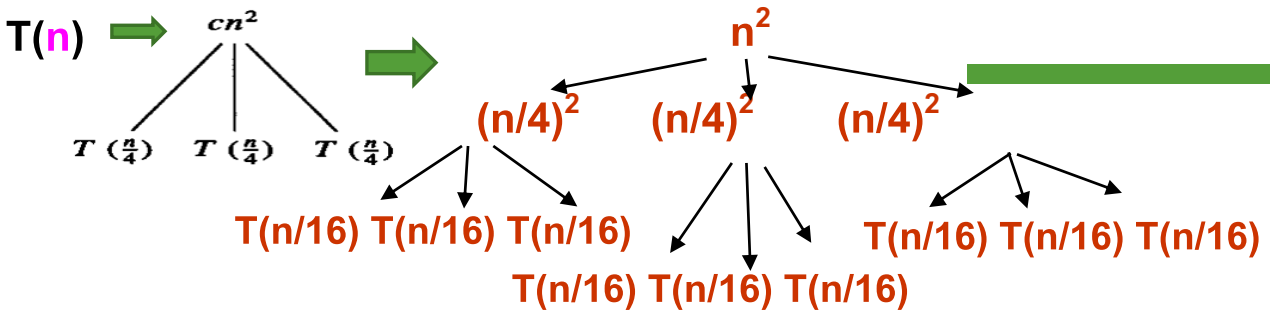
= (k+1) (n) //รวม level 0

= (log n +1) (n)

= n log n + n = O(n log n)

สมการ Recurrence $T(n) = 3T(n/4) + n^2$, $T(1) = 1$

ตัวอย่าง



level ➔	จำนวนโหนด x เวลาที่ใช้		
0 , $T(n)$	➔ $1 \times n^2$		$= (3/16)^0 n^2$
1 , $T(n/4)$	➔ $3 \times (n/4)^2$	$=(3/16)\, n^2$	$= (3/16)^1 n^2$
2, $T(n/16)$	➔ $9 \times (n/16)^2$	$=(3^2/16^2)\, n^2$	$= (3/16)^2 n^2$
....			....
$i=k-1, T(n/4^{k-1})$	➔ $3^{k-1} \times (n/4^{k-1})^2$		$= (3/16)^{k-1} n^2$
$i=k, T(1) = T(n/4^k)$	➔ $3^k \times T(1) = 3^{\log_4 n} \times 1 = n^{\log_4 3}$		

รอบที่ 0 ... i-1 รอบสุดท้าย i= k

$$
\begin{aligned}
T(n) &= \sum_{i=0}^{k-1} \left(\frac{3}{16}\right)^i n^2 + \left(n^{\log_4 3}\right) T(1) \\
&= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i n^2 + \left(n^{\log_4 3}\right) T(1) \\
&\le \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i n^2 + \left(n^{\log_4 3}\right) \\
&\le \frac{1}{1-\frac{3}{16}} n^2 + \left(n^{\log_4 3}\right) \\
T(n) &= O(n^2)
\end{aligned}
$$

- การหา<u>จำนวน level</u>

  รอบสุดท้ายคือ $T(1) = T(n/4^k)$ ดังนั้น $n = 4^k \Rightarrow k = \log_4 n$
- หาจำนวนโหนด
  - แต่ละ level ใน tree, level i, จะมีจำนวน 3^i โหนด
  - รอบสุดท้ายจะมี 3^k โหนด

    ➔ $3^{\log_4 n}$

- คำนวณ T(n) นำทุกระดับมาบวกกัน

$T(n) = (3/16)^0 n^2 + (3/16)^1 n^2 + (3/16)^2 n^2 + \ldots + (3/16)^{k-1} n^2 + n^{\log_4 3}$

$= ((3/16)^0 + (3/16)^1 + (3/16)^2 + \ldots + (3/16)^{k-1}) n^2 + n^{\log_4 3}$

ใช้ inverse harmonic series $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ for $|x| < 1$

Analyzing recursions with tree method

For integers *a* and *b*, $a \leq b$, $$\sum_{i=a}^{b} 1 = b - a + 1$$

สูตรที่ใช้บ่อย ๆ

For $n \geq 0$,

1) Linear Series

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

(Arithmetic Series)

$$\sum_{i=1}^{n-1} i = 1 + 2 + 3 + \ldots + n - 1 = \frac{(n-1)(n)}{2}$$

2) Quadratic Series: $$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

3) Cubic Series: $$\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

Analyzing recursions with tree method

สูตรที่ใช้บ่อย ๆ

Geometric series $\sum_{i=0}^{n} x^i = \frac{x^{n+1} - 1}{x - 1}$ for real $x > 1$

Inverse harmonic series

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x} \quad \text{for } |x| < 1$$

When x $= 1/2$ the series is

$$1 + 1/2 + 1/4 + \ldots = \frac{1}{1/2} = 2$$

Analyzing recursions with Master Method

- จาก recurrences ของ T(n)ในรูปแบบ $T(n) = aT(n/b) + f(n)$ โดยที่
  - ✓ $a \geq 1$, $b > 1$ เป็นค่าคงที่ และ f(n) เป็นฟังก์ชันเวลาของอัลกอริทึมสำหรับ input size
- มีวิธีหาคำตอบของ T(n) แยกเป็น 3 กรณี คือ

Case 1. คือ f(n) มีค่าน้อยกว่า $n^{\log_b a - \varepsilon}$ คำตอบของ $T(n)$ คือ $n^{\log_b a}$

เช่น $T(n) = 16T(n/4) + n$

$a = 16$, $b = 4$, $f(n) = n$

$n^{\log_b a} = n^{\log_4 16} = n^2$.

พบว่า $f(n) = n < n^{\log_b a}$ *โดยที่* $f(n) = n = O(n^{\log_b a - \varepsilon}) = O(n^{2-\varepsilon})$, และ $\varepsilon > 0$

นั่นคือ $\varepsilon = 1$ ดังนั้นเป็น Case 1.

คำตอบของ $T(n)$ คือ $n^{\log_b a} = n^2$

Analyzing recursions with Master Method

- จาก recurrences ของ T(n)ในรูปแบบ $T(n) = aT(n/b) + f(n)$ โดยที่
  - ✓ $a \geq 1$, $b > 1$ เป็นค่าคงที่ และ f(n) เป็นฟังก์ชันเวลาของอัลกอริทึมสำหรับ input size
- มีวิธีหาคำตอบของ T(n) แยกเป็น 3 กรณี คือ

Case 2. (เท่ากัน) คือ $f(n) = n^{\log_b a}$, คำตอบของ $T(n)$ คือ $n^{\log_b a}\log n$.

เช่น $T(n) = T(3n/7) + 1$

$a = 1$, $b=7/3$, $f(n) = 1$

$n^{\log_b a} = n^{\log_{(7/3)} 1} = n^0 = 1$

พบว่า $f(n) = 1$ *เท่ากับ* $n^{\log_b a} = 1$ ➔ ดังนั้นเป็น Case 2.

คำตอบของ $T(n)$ คือ $n^{\log_b a} \log n = \log n$

Analyzing recursions with Master Method

- จาก recurrences ของ T(n)ในรูปแบบ $T(n) = aT(n/b) + f(n)$ โดยที่
  - ✓ $a \geq 1$, $b > 1$ เป็นค่าคงที่ และ f(n) เป็นฟังก์ชันเวลาของอัลกอริทึมสำหรับ input size
- มีวิธีหาคำตอบของ T(n) แยกเป็น 3 กรณี คือ

Case 3. คือ f(n) มีค่ามากกว่า $n^{\log_b a}$

เช่น $T(n) = 3T(n/4) + n \log n$

$a = 3$, $b=4$, $f(n) = n \log n$

$n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$

พบว่า $f(n) = n \log n$ *มีค่ามากกว่า* $n^{\log_4 3}$

โดยที่ $n^{\log_4 3 + \varepsilon} = n$ *เมื่อ* $\varepsilon \approx 0.2$

ทำให้ $f(n) = \Omega(n^{\log_b a + \varepsilon}) = \Omega(n)$

n เป็น lower bound ของ $n \log n$

$\Rightarrow$ Case 3.

และสำหรับ large n,

จะหาค่า $c < 1$ และทำให้สมการต่อไปนี้เป็นจริงได้

$$a\, f(n/b) = 3\,(n/4 \log n/4) \leq c\,(n \log n)$$

$$= (3/4)\, n \log n - (3/4)\, n \log 4 \leq c\,(n \log n)$$

$$= (3/4)\, n \log n - (3/2)\, n \leq c\,(n \log n)$$

พบว่าสามารถเป็นจริงได้เมื่อ $c = 3/4$ *โดยที่* $c < 1$

คำตอบของ $T(n)$ คือ $f(n) = n \log n$

Analyzing recursions with Master Method

ตัวอย่างที่ใช้ **Master Method ไม่ได้**

$T(n) = 2T(n/2) + n \log n$

$a = 2,\ b=2,\ n^{\log_b a} = n^{\log_2 2} = n$

$f(n) = n \log n$, and $f(n)$ *มากกว่า* $n^{\log_b a}$

ไม่สามารถหา $\varepsilon > 0$ ที่ทำให้ $f(n) = \Omega(n^{\log_b a+\varepsilon}) = \Omega(n^{1+\varepsilon})$

(เพราะว่า ratio ของ $\log n$ น้อยกว่า n^{ε})

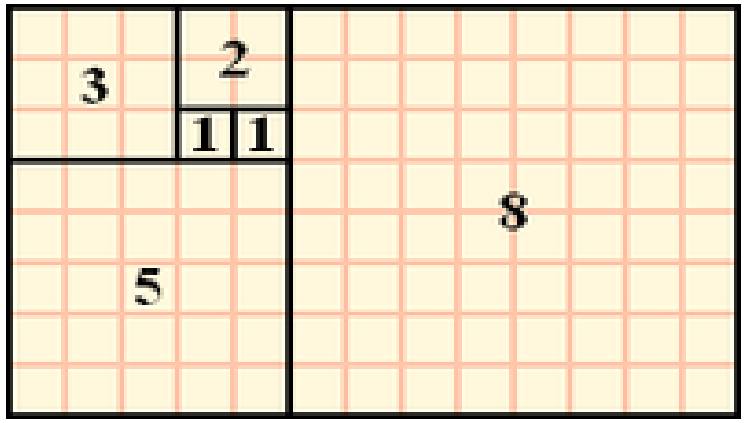
Fibonacci number

- Fibonacci number เป็นตัวอย่างของปัญหาที่มีการเรียกหาคำตอบของปัญหาย่อยซ้ำ ๆ กันมาก ๆ

- จำนวนฟีโบนัชชี / เลขฟีโบนัชชี คือ ชุดตัวเลขที่อยู่ในลำดับจำนวนเต็มในลักษณะต่อไปนี้
  - ✓ 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, …

- ซึ่ง Fibonacci number สามารถเขียนได้ในรูป

$F_n = F_{n-1} + F_{n-2}$ โดยที่ $F_0 = 0, F_1 = 1$



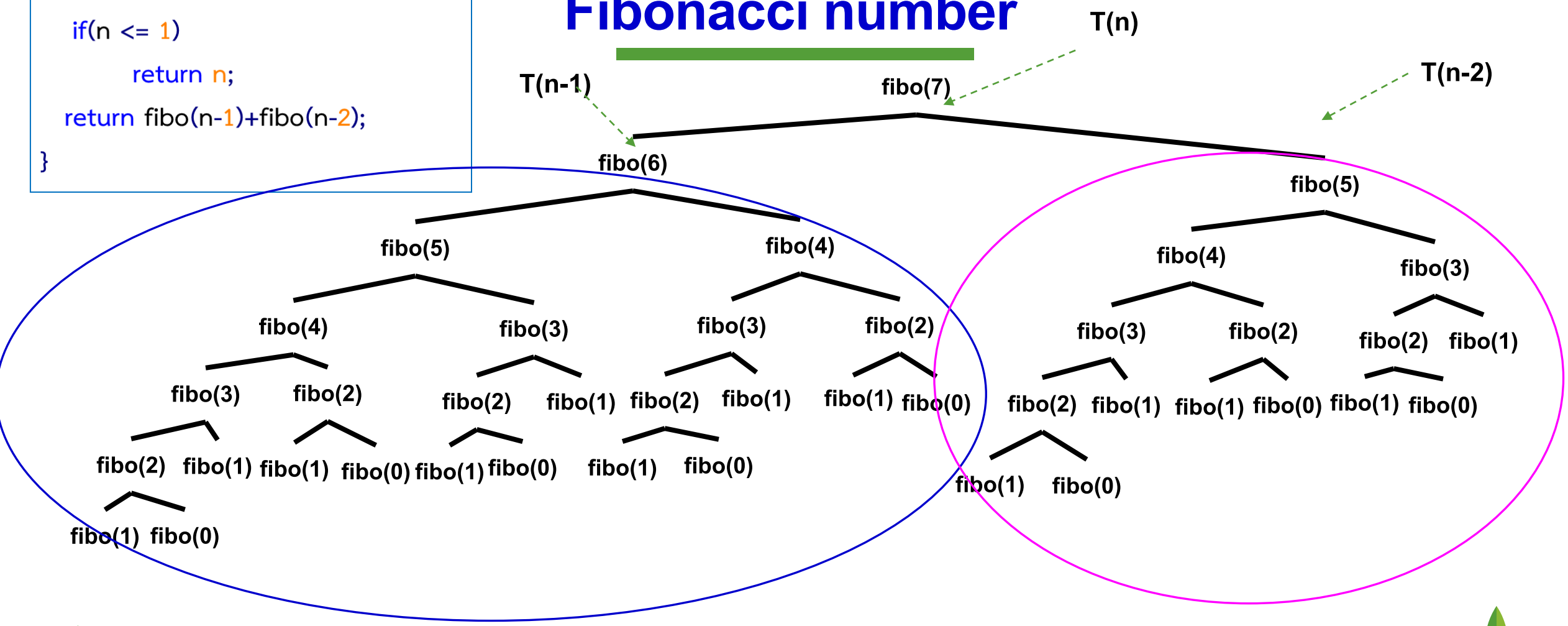
เมื่อเขียนเป็น recursive จะได้

```
int fibo(int n){
    if(n <= 1)
        return n;
    return fibo(n-1)+fibo(n-2);
}
```

Fibonacci number

เมื่อเขียนเป็น recursive จะได้

```
int fibo(int n){
    if(n <= 1)
        return n;
    return fibo(n-1)+fibo(n-2);
}
```



- $T(n) = T(n-1) + T(n-2) + C \approx 2^{n-1} + 2^{n-2} + O(1) \approx O(2^n)$

แบบฝึกหัด

ให้หาค่า T(n) จากสมการที่กำหนดให้ ด้วยวิธี Recursion-tree และ Master Theorem

1. $T(n) = 4T(n/4) + 1, T(1)=1$
2. $T(n) = 3T(n/3) + n, T(1) = 1$
3. $T(n) = 2T(n/3) + n^2, T(1)=1$
4. $T(n) = 4T(n/2) + n , T(1)=1$
5. $T(n) = 4T(n/2) + n^2, T(1)= 1$

Hint : solution

1. n
2. $n \log n$
3. n^2
4. n^2
5. $n^2 \log n$